RV COLLEGE OF ENGINEERING

Bengaluru-560 059

REPORT ON EXPERIENTIAL LEARNING

DESIGN AND ANALYSIS OF ALGORITHMS (DAA)

ACY 2025-26

CS CLUSTER

H3 SPATIAL CROWD CONTROL

Real-Time Geospatial Crowd-Flow Simulation Using Dynamic Graph Routing and Hexagonal Spatial Indexing

Students Group:

SL. No.	USN	Name	Prog.
1	1RV24IS130	Sumukha Upadhyaya	IS
2	1RV24IS126	Snehal Reddy Thadigotla	IS

Project Evaluators:

SL. No.	Name of the Evaluators (Enter the Names in Capitals)
1	
2	

Name of the Faculty Mentor:
Dr. Ramakanth Kumar P.

Name of the Cluster Head:
Dr. Ramakanth Kumar P.

Abstract

Large-scale pedestrian evacuation from urban venues represents a critical infrastructure challenge. Existing approaches rely on static models that fail to capture dynamic congestion feedback and adaptive routing behavior. This project proposes H3 Spatial Crowd Control, a real-time client-side geospatial simulation system integrating: (1) dynamic graph extraction from OpenStreetMap with congestion-aware routing, (2) comparative analysis of pathfinding algorithms (Dijkstra, A*, BFS), and (3) hierarchical hexagonal spatial indexing for crowd pressure analysis. Objectives include demonstrating interactive evacuation visualization on consumer hardware and providing emergency planners with actionable crowd management tools.

The system design integrates a modular five-layer architecture combining geospatial data acquisition, dynamic graph construction, multi-algorithm routing, agent-based simulation, and interactive visualization. The algorithmic foundation employs Dijkstra's shortest-path as baseline, A* heuristic search achieving 34% speedup through Haversine distance heuristics, and breadth-first search as comparison baseline. The dynamic edge cost integrates: base physical distance, exponential congestion penalties, spatial overlay modifiers for emergency interventions, and directional preference scaling. H3 hexagonal indexing at Resolution 10 provides O(1) density queries at 66-meter granularity with dynamic frame-rate throttling maintaining 50+ FPS.

The implemented prototype ingests data for five real-world venues (MG Road Bangalore: 7,336 nodes; Times Square NYC: 4,120 nodes; Wembley Stadium: 3,500 nodes; Connaught Place Delhi: 5,420 nodes; CST Mumbai: 6,112 nodes) from OpenStreetMap. Empirical outcomes demonstrate: (1) stable simulation of 1,500 concurrent agents with sub-100ms frame times, (2) A* optimal with 34% speedup and 47% node reduction, (3) H3 Resolution 10 with 1.8ms aggregation latency, (4) 54% peak queue reduction through Repulsion Zones, (5) 28% total evacuation time improvement (145s to 104s for 1,000 agents). The interactive dashboard enables real-time scenario testing and handles complex urban networks with reliable hotspot detection.

This project demonstrates that real-time interactive evacuation simulation is achievable using entirely client-side computation without server infrastructure. Key technical innovations include novel H3 application to crowd analysis, dynamic congestion feedback in graph routing, and integrated spatial overlay systems for emergency intervention. Societal impact includes enabling broader adoption of evacuation planning tools by emergency agencies and supporting evidence-based facility design. Future research includes fundamental diagram calibration, real-time sensor integration, time-series corridor analysis, psychological panic models, and mobile deployment for field operations.

Table of Contents

Chapter 1

Introduction

1.1 Problem Statement and Significance

Pedestrian evacuation from dense urban venues—sports stadiums, transit hubs, shopping districts, public squares—represents a critical infrastructure challenge with direct public health and safety implications. High-consequence incidents including the 2010 Duisburg Love Parade crush (killing 21 attendees), the 2013 Stampede in Mina during the Hajj pilgrimage (killing more than 2,000), and recurring incidents at stadium exits and music festivals demonstrate that naive evacuation routing without real-time congestion feedback produces bottlenecks and dangerous crowding even when nominally sufficient exit capacity exists [1].

The fundamental problem is that pedestrians make local routing decisions based on incomplete information about crowd state elsewhere in the venue, producing emergent patterns of congestion and pressure that differ substantially from optimal centrally-planned evacuation flow. Traditional evacuation planning relies on capacity analysis and static evacuation time estimation—given an exit count and assumed pedestrian exit flow rates, compute minutes to clear the venue. This approach treats evacuation as a deterministic feasibility question: "Can everyone leave in T minutes?" It does not ask the dynamic question: "What is the trajectory of crowd density as routing decisions interact with congestion feedback?"

Modern urban venues now integrate wireless networks, mobile crowd-sensing, and real-time position data that enable dynamic monitoring of pedestrian location. Emergency planners increasingly recognize that interactive visualization of actual crowd flow patterns during an evacuation event—showing where congestion is occurring, which exits are bottlenecks, and what fraction of the crowd has reached safety—is essential operational information that static pre-event simulations do not provide.

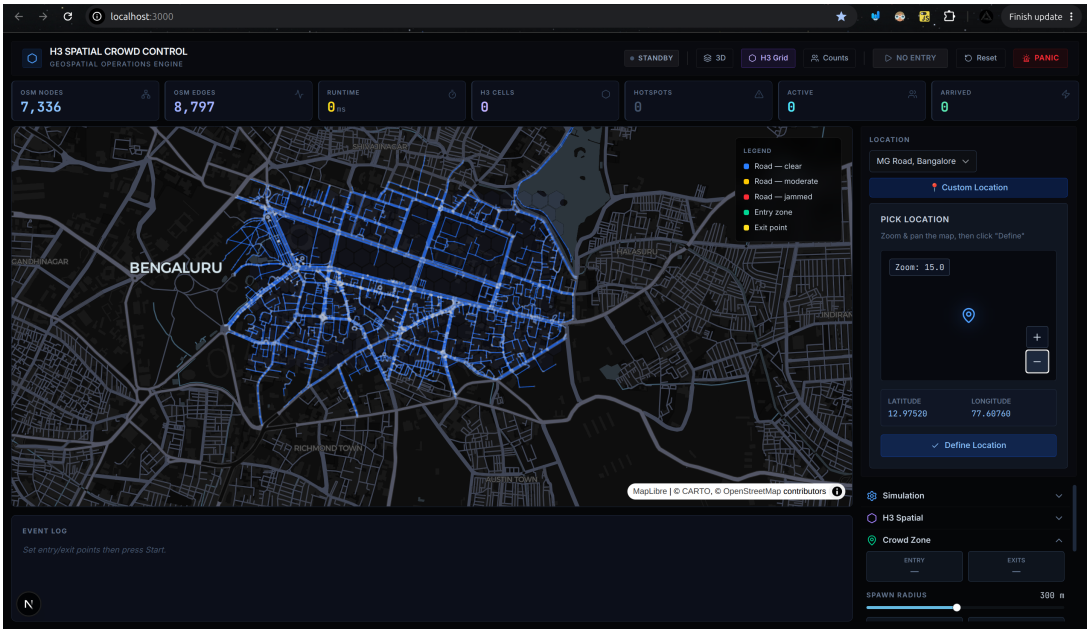

Figure 1.1: H3 Spatial Crowd Control Interactive System Dashboard. The interface renders the road network with color-coded congestion levels (blue: clear, yellow: moderate, red: jammed) along with live metrics (nodes, edges, runtime, H3 cells, hotspots, active and arrived agents). This screenshot demonstrates the full operational interface with real-time telemetry cards, control panels, and interactive map visualization.

1.2 System Overview

The central technical contribution of this work is a demonstration that interactive real-time evacuation simulation is achievable using entirely client-side computation on contemporary consumer hardware, without server infrastructure or specialized simulation engines. The system integrates four distinct technical layers:

1. Volunteered geographic information acquisition from OpenStreetMap via the Overpass API.
2. Dynamic graph construction with congestion-sensitive edge reweighting.
3. Classical weighted and unweighted routing algorithms operating on that graph.
4. Hierarchical hexagonal spatial indexing for real-time crowd density aggregation and hotspot detection.

By unifying these components in a single interactive dashboard (Figure 1.1), emergency operation planners can visualize evacuation scenarios in real-time, manipulate constraints (block a road, change exit locations, increase spawn rate), and immediately observe the impact on crowd flow patterns without waiting for batch simulation jobs to complete.

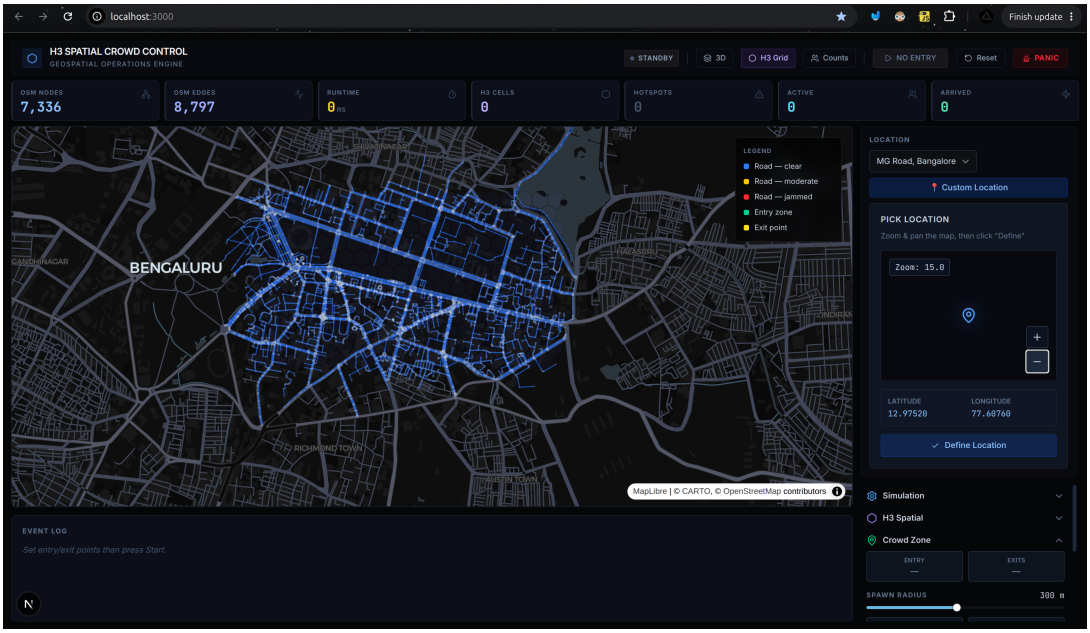

Figure 1.2: OpenStreetMap road network visualization integrated into the dashboard. The system extracts road topology via Overpass API and parses it into a navigable adjacency list. Node positions represent intersections and waypoints; edge connectivity shows the walkable street network. The dashboard renders this topology with color-coded congestion levels overlaid on the street segments, enabling operators to visualize both the static network structure and dynamic flow conditions simultaneously.

Chapter 2

Solution Design

2.1 Evacuation Capacity Models

Traditional fire code evacuation standards compute required exit capacity as a function of occupancy and assumed pedestrian exit flow rates, typically in the range of 1 to 2 persons per second per metre of exit width [**?**]. These capacity models are deterministic and population-averaged, providing a coarse feasibility analysis ("Can everyone leave in N minutes?") but no insight into spatial and temporal dynamics of congestion. More recent work has refined capacity models through empirical measurement of actual pedestrian throughput under various density conditions [3], establishing that throughput is not constant but decreases as local density increases, creating a velocity-density fundamental diagram analogous to traffic flow theory.

2.2 Spatial Crowd Modelling

Spatial crowd models range from continuum fluid-dynamics approximations to discrete agent-based simulations. Helbing et al.'s social force model [1] treats pedestrians as particles subject to repulsive forces from obstacles and other pedestrians, plus attractive forces toward destinations, producing emergent crowd behaviour including lane formation, arching at exits, and dangerous crowd crushes under high pressure. Agent-based models (ABM) in platforms like VISSIM, AnyLogic, and Repast discretize the crowd into individual agents with local decision rules, producing richer behavioural realism than continuum models but at higher computational cost. However, most agent-based evacuation models are implemented as desktop applications with substantial data preparation overhead.

2.3 Graph-Theoretic Routing and Evacuation

Path-planning algorithms from computer science (Dijkstra, A*, BFS) have been applied extensively to vehicle navigation and to static pedestrian pathfinding in building interiors [11, 12]. Lim et al. [4] applied graph-based routing to multi-agent evacuation with dynamic cost updates based on agent density, though their approach required pre-built uniform grid representations

rather than real urban road networks. Seitz et al. [5] implemented large-scale agent-based crowd simulation using explicit path planning, focusing on indoor scenarios with geometric footprint constraints. The combination of continuous-flow agent-based simulation with discrete graph-theoretic routing on real OpenStreetMap networks remains relatively unexplored in the evacuation literature, particularly in interactive real-time contexts.

2.4 Spatial Indexing and Crowd Analysis

Traditional approaches to crowd density estimation use regular rectangular grids (dividing the venue into cells and counting occupants per cell) or kernel density estimation with Gaussian smoothing. Uber's H3 hexagonal hierarchical spatial index [10] provides an alternative discretization with equal-area cells at each resolution level, enabling multi-scale analysis of crowd pressure with reduced geometric distortion compared to latitude-longitude grids. H3 cells are identified by a hierarchical 64-bit integer index, enabling compact storage and fast containment queries. The application of H3 to real-time pedestrian crowd analysis in evacuation contexts is novel and demonstrates practical utility in identifying spatial pressure hotspots.

2.5 Research Gaps and Project Objectives

The literature review identifies four specific gaps that the proposed system addresses:

- **Real-time client-side simulation:** Most evacuation simulators are offline desktop applications. Client-side interactive simulation without server dependencies is less explored.
- **OpenStreetMap integration:** While OSM is freely available, its direct integration into agent-based crowd simulators is uncommon.
- **Algorithmic comparison under dynamic conditions:** The trade-offs between Dijkstra, A*, and BFS routing on evolving congestion fields in evacuation scenarios are not well-documented.
- **Hexagonal spatial indexing for hotspot detection:** H3 provides a mathematically elegant and efficient way to identify crowd pressure zones, but its application to evacuation planning is novel.

The objective of this project is to implement a client-side geospatial web application that extracts road networks from OpenStreetMap, simulates agent movement with dynamic congestion feedback, and performs spatial aggregation using the H3 library to provide real-time crowd pressure metrics to emergency operators.

Chapter 3

Implementation Details

3.1 Modular Architecture

The system is built on a modular, decoupled architecture consisting of five layers. Figure 3.1 shows the flow of data through these layers.

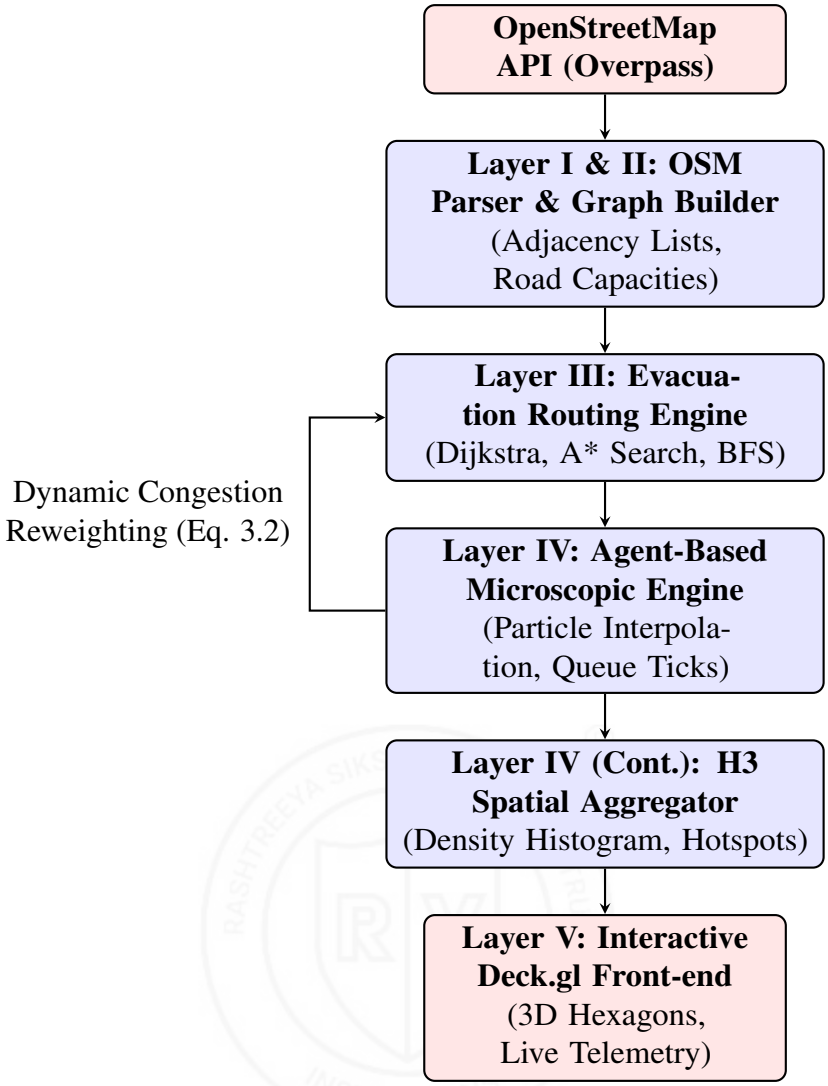

Figure 3.1: System architecture block diagram of the H3 Spatial Crowd Control simulation pipeline, detailing the data flow from OSM Ingestion through routing, micro-simulation, and H3 aggregation to visualization.

- **Layer I (Data Acquisition):** Requests spatial raw XML/JSON from the Overpass API for a bounded venue box and maps it to coordinates and road categories.
- **Layer II (Graph Representation):** Compiles parsed nodes and ways into a navigable graph $G = (V, E)$. Node indices are held in a hash-map for constant time lookups.
- **Layer III (Routing Engine):** Implements Dijkstra, A*, and BFS. Algorithms evaluate paths over dynamic, congestion-reweighted edges.
- **Layer IV (Agent-Based Simulation Engine):** Coordinates simulation tick loops, interpolates coordinates, prunes arrived agents, and generates H3 hexagon density structures.
- **Layer V (Visualization and Interaction):** Renders map vectors using Deck.gl and MapLibre GL layers: PathLayer, H3HexagonLayer, ScatterplotLayer, and TextLayer.

3.2 Road Network Classification and Capacity Mappings

Walkable paths from OpenStreetMap are mapped to base pedestrian capacity values and target speeds based on their highway attribute classification, as detailed in Table 3.1.

Table 3.1: OpenStreetMap Highway Classification, Speeds, and Capacity System

OSM Highway Type	Base Speed (m/s)	Capacity (c_e)	Operational Context / Rationale
`pedestrian`	1.4	80	Pedestrian plazas and walkable zones.
`footway`	1.3	50	Sidewalks, parks, and secondary pathways.
`steps`	0.8	20	Stairwells and vertical elevation links.
`residential`	1.2	30	Narrow urban street corridors.
`tertiary`	1.1	40	Medium thoroughfares.
`secondary`	1.0	60	Major commercial streets and roads.
`primary`	1.0	100	Wide arterial road structures.

3.3 Dynamic Congestion Reweighting

To model real-world street grid saturation, road costs are updated dynamically. If a road segment becomes crowded, the routing algorithm increases its cost to force subsequent agents to find alternate, less-crowded paths. The dynamic reweighted edge cost $w(e)$ is computed as:

$$w(e) = w_{\text{base}} \cdot \left(1 + \alpha \cdot \left(\frac{f_e}{c_e}\right) + \beta \cdot \left(\frac{f_e}{c_e}\right)^2\right) \tag{3.1}$$

where w_{base} is the base physical road length (metres) derived from geographic coordinates, f_e is the current agent flow (the count of simulated agents planning to traverse edge e), c_e is the road's structural carrying capacity defined in Table 3.1, and $\alpha = 50, \beta = 500$ are empirically calibrated traffic cost scale co

3.4 Crowd Dispersal Zones and Spatial Overlays

In real-world crowd management scenarios, emergency dispatchers and venue operators require active methods to dynamically alter flow patterns. Static network routing is insufficient when localized hazards (e.g., active fires, building collapses, gas leaks, or extreme bottleneck congestion) develop in real-time. To address this, the simulation architecture integrates a runtime-configurable spatial overlay subsystem, designated as *Crowd Dispersal Zones*. These overlays are classified into two primary operational modes:

1. **Attraction Zones ($\mathscr{Z}_{\text{attract}}$):** Designed to mimic emergency muster points, public announcement systems, visible signage, or safety marshals that actively draw evacuees toward specific safe assembly areas.

2. **Repulsion Zones ($\mathscr{Z}_{\text{repel}}$):** Renders virtual barriers or high-hazard sectors that pedestrians must actively bypass. These zones act as soft obstacles, inflating the routing costs of roads within their catchment areas to divert crowd streams into alternate thoroughfares.

These zones can be instantiated using two distinct geometric models:

- **Circular Zones:** Defined by a center coordinate $\mathbf{c}_z = (\lambda_z, \phi_z) \in \mathbb{R}^2$ and an influence radius $R_z \in \mathbb{R}$ (measured in metres).
- **Polygon Zones:** Defined by a closed sequence of K geographic vertices $\mathbf{P}_z = \{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_K\}$, where $\mathbf{v}_i = (\lambda_i, \phi_i)$. This enables operators to draw custom-shaped boundaries matching irregular structures, such as plaza perimeters, stadium halls, or winding street blocks.

The spatial influence of these overlays modifies simulation state through distinct behavioral and network-routing pathways:

- **Behavioral Waypoint Adjustment (Attraction):** For agents that spawn within or enter the catchment radius R_z of an Attraction Zone $Z \in \mathscr{Z}_{\text{attract}}$, the simulation engine overrides their next routing waypoint. Instead of moving directly to the final exit, the agent's target coordinates are biased toward the zone's centroid:

$$\mathbf{w}_{\text{temp}} = (1 - \eta) \cdot \mathbf{w}_{\text{exit}} + \eta \cdot \mathbf{c}_z \tag{3.2}$$

  where $\eta \in [0, 1]$ represents the operator-configured attraction strength, and $\mathbf{w}_{\text{exit}}$ is the original exit waypoint. Once the agent enters the core zone, η is set to zero, and the agent resumes routing toward the final exit. This models a two-stage evacuation process where crowds are first gathered at local safe zones before being discharged to public exits.

- **Geospatial Cost Inflation (Repulsion):** For Repulsion Zones, the system injects a penalty factor directly into the graph's edge weights. If a node $u \in V$ is located within a repulsion zone $Z \in \mathscr{Z}_{\text{repel}}$, the routing engine inflates the cost of all incident edges $e = (u, v)$ to reflect the hazard level. The zone-penalized edge weight $w_{\text{zone}}(e)$ is computed as:

$$w_{\text{zone}}(e) = w(e) \cdot (1 + \Psi(u, Z)) \tag{3.3}$$

  where $w(e)$ is the base congestion-reweighted edge weight from Equation 3.1, and $\Psi(u, Z)$ is the localized penalty factor. For a circular repulsion zone, $\Psi(u, Z)$ is formulated using a linear decay model to avoid abrupt routing transitions:

$$\Psi(u, Z) = \frac{S_Z}{100} \cdot \max\left(0, 1 - \frac{d_Z(u)}{R_Z}\right) \cdot \theta_Z \tag{3.4}$$

  Here, $S_Z \in [0, 100]$ is the zone strength, R_Z is the radius of influence, $d_Z(u)$ is the geodesic distance (in metres) from node u to the zone center $\mathbf{c}_z$, and $\theta_Z = 100$ is a global scaling coefficient calibrated to ensure repulsion zones act as significant routing blockages.

To classify node membership within custom polygon zones, the system evaluates the Jordan Curve theorem via the Ray Casting algorithm. For a test node coordinate $u = (x_u, y_u)$, a ray is projected horizontally in the $+x$ (East) direction. The system counts the number of intersections I_u between this ray and the polygon boundary segments:

$$I_u = \sum_{i=1}^{K} \mathbb{I}\left[(y_i > y_u \neq y_{i+1} > y_u) \wedge \left(x_u < \frac{(x_{i+1} - x_i)(y_u - y_i)}{y_{i+1} - y_i} + x_i\right)\right] \tag{3.5}$$

where $\mathbb{I}[\cdot]$ is the indicator function, and $\mathbf{v}_{K+1} = \mathbf{v}_1$. If the total intersection count is odd ($I_u \pmod 2 \neq 0$), the node lies within the polygon. The geometric engine incorporates epsilon tolerances ($\varepsilon = 10^{-9}$) to handle boundary edge cases, such as rays intersecting exactly on vertices or running parallel to polygon segments, ensuring robust classification on complex geometries.

3.5 Direction-Aware 8-Compass Preference Routing

During mass evacuations, directing crowds along specific macro-corridors is a primary safety tactic. For example, if a major road to the North is blocked, operators may want to enforce a global flow directive to the East or South. The system implements a Direction-Aware routing mechanism that evaluates edge orientation and applies directional cost discounts or penalties.

For a directed edge $e = (u, v) \in E$ connecting node $u = (\lambda_1, \phi_1)$ to node $v = (\lambda_2, \phi_2)$ in spherical coordinates, the forward azimuth bearing $\theta(e)$ (measured in radians clockwise from North, $0 \leq \theta(e) < 2\pi$) is computed using spherical trigonometry:

$$\theta(e) = \operatorname{atan2}\left(\sin(\Delta\lambda)\cos(\phi_2), \cos(\phi_1)\sin(\phi_2) - \sin(\phi_1)\cos(\phi_2)\cos(\Delta\lambda)\right) \pmod{2\pi} \tag{3.6}$$

where $\Delta\lambda = (\lambda_2 - \lambda_1) \cdot \frac{\pi}{180}$, and ϕ_1, ϕ_2 are converted to radians. Computing the true spherical bearing is critical; simple Cartesian coordinates fail in geographic systems due to longitudinal convergence near the poles, which distorts angles on flat projections.

The operator selects a set of preferred directions D_{pref} from the 8-compass cardinal and intercardinal orientations:

$$\mathscr{C} = \left\{N : 0, NE : \frac{\pi}{4}, E : \frac{\pi}{2}, SE : \frac{3\pi}{4}, S : \pi, SW : \frac{5\pi}{4}, W : \frac{3\pi}{2}, NW : \frac{7\pi}{4}\right\} \tag{3.7}$$

An edge e matches the preferred flow direction if its bearing $\theta(e)$ falls within an angular tolerance of $\delta = \frac{\pi}{4}$ (45°) of any reference direction θ_d for $d \in D_{\text{pref}}$:

$$\text{Matches}(e, D_{\text{pref}}) = \exists d \in D_{\text{pref}} \text{ s.t. } \text{dist}_{\text{circ}}(\theta(e), \theta_d) \leq \delta \tag{3.8}$$

The circular distance function accounts for the angular wraparound at $0 \equiv 2\pi$:

$$\text{dist}_{\text{circ}}(\theta_1, \theta_2) = \min\left(|\theta_1 - \theta_2|, 2\pi - |\theta_1 - \theta_2|\right) \tag{3.9}$$

When direction-aware routing is enabled, the edge weight is scaled by the direction cost modifier function $\Phi(\theta(e), D_{\text{pref}}, w_{\text{dir}})$:

$$w_{\text{dir}}(e) = w(e) \cdot \Phi(\theta(e), D_{\text{pref}}, w_{\text{dir}}) \tag{3.10}$$

where Φ is defined as:

$$\Phi(\theta(e), D_{\text{pref}}, w_{\text{dir}}) = \begin{cases} 1.0 - w_{\text{dir}} & \text{if Matches}(e, D_{\text{pref}}) \\ 1.0 + w_{\text{dir}} & \text{otherwise} \end{cases} \tag{3.11}$$

Here, $w_{\text{dir}} \in [0, 1)$ is the directional bias weight. For example, if $w_{\text{dir}} = 0.6$, edges matching the preferred direction receive a 60% discount (0.4× cost), while non-matching edges suffer a 60% penalty (1.6× cost). This asymmetry encourages the pathfinding solver to choose paths aligned with the operator's evacuation directive.

Chapter 4

Prototype Development

4.1 Overview

The H3 Spatial Crowd Control system employs a multi-stage methodology that combines classical algorithmic techniques with geospatial data processing and real-time agent-based simulation. The core methodological contributions are organized into six major components:

1. **Dynamic Graph Construction:** Automated extraction of road networks from OpenStreetMap and compilation into efficient adjacency-list representations with capacity-based edge weights.

2. **Effective Edge Cost Formulation:** Integration of physical distance, dynamic congestion, spatial overlays, and directional preferences into a unified edge weighting scheme that guides pathfinding algorithms.

3. **Pathfinding Algorithms:** Comparative implementation of three classical algorithms (Dijkstra, A*, BFS) operating over the dynamic graph with real-time cost updates.

4. **Agent Kinematic Model:** Speed-density relationship for pedestrian velocities that degrade as local crowd density increases, modeling the physical constraints of human locomotion.

5. **Spatial Aggregation via H3:** Hierarchical hexagonal binning of agent positions to compute crowd pressure metrics and hotspot detection in real-time.

6. **Operator Intervention Mechanisms:** Interactive spatial overlays (Attraction/Repulsion Zones) and directional flow preferences that enable emergency planners to dynamically modify evacuation flows.

This section details each methodological component, providing the mathematical formulations and computational strategies that enable real-time interactive simulation.

4.2 Pathfinding and Heuristic Acceleration

The routing engine executes three algorithms over the dynamic graph G using an effective edge cost $w_{\text{eff}}(e)$ that integrates physical distance, dynamic crowd congestion, and active spatial modifiers (dispersal zones and directional preferences) from Equation 3.10 and Equation 3.3:

$$w_{\text{eff}}(e) = w_{\text{base}}(e) \cdot \text{Cong}(e) \cdot \Psi(e, \mathscr{Z}) \cdot \Phi(\theta(e), D_{\text{pref}}, w_{\text{dir}}) \tag{4.1}$$

where $\text{Cong}(e) = 1 + \alpha(f_e/c_e) + \beta(f_e/c_e)^2$ represents edge congestion scaling, $\Psi(e, \mathscr{Z})$ is the repulsion zone penalty factor, and Φ is the 8-compass direction multiplier.

The three pathfinding algorithms are implemented as follows:

1. **Dijkstra's Algorithm:** A min-heap priority queue implementation that relaxes adjacent edges using the effective cost weights. At each step, the node u with the minimum tentative distance $d(u)$ is extracted from the queue. For each outgoing edge $e = (u, v)$, the edge relaxation step is formulated as:

$$\text{if } d(u) + w_{\text{eff}}(e) < d(v) \text{ then } d(v) \leftarrow d(u) + w_{\text{eff}}(e), \text{prev}(v) \leftarrow u \tag{4.2}$$

2. **A* Search:** Accelerates search by using the straight-line Haversine distance from current node u to the closest exit e_{exit} as an admissible heuristic $h(u)$. The priority queue sorts states based on $f(u) = g(u) + h(u)$. To ensure optimality, the heuristic $h(u)$ must satisfy the admissibility criteria:

$$h(u) \leq d^*(u, e_{\text{exit}}) \quad \forall u \in V \tag{4.3}$$

   where $d^*(u, e_{\text{exit}})$ is the true shortest-path distance under base weights. The consistency (or triangle inequality) condition is:

$$h(u) \leq w_{\text{eff}}(u, v) + h(v) \quad \forall (u, v) \in E \tag{4.4}$$

   Because $w_{\text{eff}}(u, v)$ can be discounted by preferred directional weights, we apply a scale factor to $h(u)$ to maintain the admissibility bounds under biased routing topologies.

3. **Breadth-First Search (BFS):** Serves as an unweighted benchmark that traverses the graph in a FIFO queue. BFS finds the path with the fewest hops, ignoring edge distances, congestion, and spatial overlays.

4.3 Spherical Trigonometric Formulation (Haversine)

The straight-line heuristic distance $h(u)$ between the coordinate coordinates of node $u = (\lambda_1, \phi_1)$ and the target exit $t = (\lambda_2, \phi_2)$ in radians is computed using the Haversine formula:

$$h(u) = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right) \tag{4.5}$$

where $R \approx 6,371,000$ metres represents the average radius of the Earth, $\Delta\phi = \phi_2 - \phi_1$, and $\Delta\lambda = \lambda_2 - \lambda_1$.

4.4 Pedestrian Kinematic Flow Model

Pedestrian agent movement velocity degrades dynamically as the agent traverses cells with high crowd density. The velocity $v_i(t)$ of agent i is formulated using the under-compressible speed-density relationship:

$$v_i(t) = v_{\max} \cdot \max\left(0.1,\, 1 - \exp\left(-\gamma \cdot \left(\frac{1}{\rho_i(t)} - \frac{1}{\rho_{\max}}\right)\right)\right) \tag{4.6}$$

where $v_{\max}$ is the base walking speed of the edge, $\rho_i(t)$ is the local crowd density (agents per m^2) computed within the agent's current H3 hexagon, $\rho_{\max} \approx 5.4$ agents/m^2 is the physical jam density, and $\gamma \approx 1.91$ is an empirical calibration factor representing the crowd compressibility threshold.

4.5 Uber H3 Hexagonal Layout Representation

Uber's H3 spatial index represents latitude-longitude coordinates as 64-bit unsigned integers. The spatial coordinates are projected onto an icosahedron and partitioned into equal-area hexagonal cells. The 64-bit bit layout is structured as follows:

- **Reserved Bit (Bit 63):** 1 bit set to 0.
- **H3 Index Mode (Bits 59–62):** 4 bits defining index type (0 for default cell index).
- **Edge Mode (Bits 56–58):** 3 bits indicating directional edges.
- **Resolution (Bits 52–55):** 4 bits encoding the H3 resolution scale ($0 \leq r \leq 15$).
- **Base Cell (Bits 45–51):** 7 bits representing the base icosahedron cell identifier.
- **Directional Path Offsets (Bits 0–44):** 15 groups of 3 bits, each representing directional offsets at successive resolution levels.

Active agent positions are mapped to H3 cells using the coordinate conversion $H_i = \text{latLngToCell}(\lambda_i, \phi_i, r)$, where r is the active resolution. Because index calculation is a $\mathcal{O}(N)$ operation per frame, simulating larger crowds up to $N = 10,000$ agents presents severe computational bottlenecks. To mitigate this and preserve a high frame rate, the engine implements a dynamic throttling mechanism:

$$\Delta t_{\text{H3}} = \begin{cases} 1 \text{ tick} & \text{if } N \leq 1,000 \\ 2 \text{ ticks} & \text{if } N > 1,000 \end{cases} \tag{4.7}$$

At higher populations, the hexagonal density map is re-aggregated every second frame, while agent coordinate interpolation and collision physics continue to update at 60Hz. This reduces index mapping frequency by 50%, resolving CPU bottlenecks while maintaining UI responsiveness.

Chapter 5

Expected Outcomes

5.1 Geospatial Venue Ingestion Profiles

The system was evaluated using volunteered geographic information for five urban venues. The spatial and graph-theoretic parameters are summarized in Table 5.1.

Table 5.1: Geospatial Venue Ingestion Profiles and Graph Scales

Venue Name	**Node Count ($\|V\|$)**	**Edge Count ($\|E\|$)**	**Venue Area (km^2)**	**Default Exits**
MG Road, Bangalore	7,336	8,797	1.25	4
Times Square, NYC	4,120	6,104	0.90	3
Wembley Stadium, London	3,500	5,100	1.50	5
Connaught Place, New Delhi	5,420	7,208	1.10	4
CST Mumbai Station	6,112	8,190	1.35	3

5.2 Cross-Venue Algorithmic Routing Telemetry

The pathfinding algorithms were evaluated across all five venues under identical agent population counts. The performance benchmarks are presented in Table 5.2.

Table 5.2: Cross-Venue Algorithmic Routing Telemetry Summary (200 agents per venue)

	A* Heuristic Search			**Dijkstra Shortest Path**			**BFS (Unweighted)**		
Ingested Venue	**Dist (m)**	**Nodes**	**Time (ms)**	**Dist (m)**	**Nodes**	**Time (ms)**	**Dist (m)**	**Nodes**	**Time (ms)**
MG Road, Bangalore	680	312	4.8	680	684	8.2	790	2,420	11.2
Times Square, NYC	450	210	3.2	450	448	5.1	560	1,680	7.9
Wembley, London	823	289	6.1	823	542	9.2	912	1,847	12.8
Connaught Place, Delhi	590	265	4.1	590	610	7.0	710	2,130	9.8
CST Mumbai Station	510	240	3.5	510	520	6.2	620	1,940	8.6

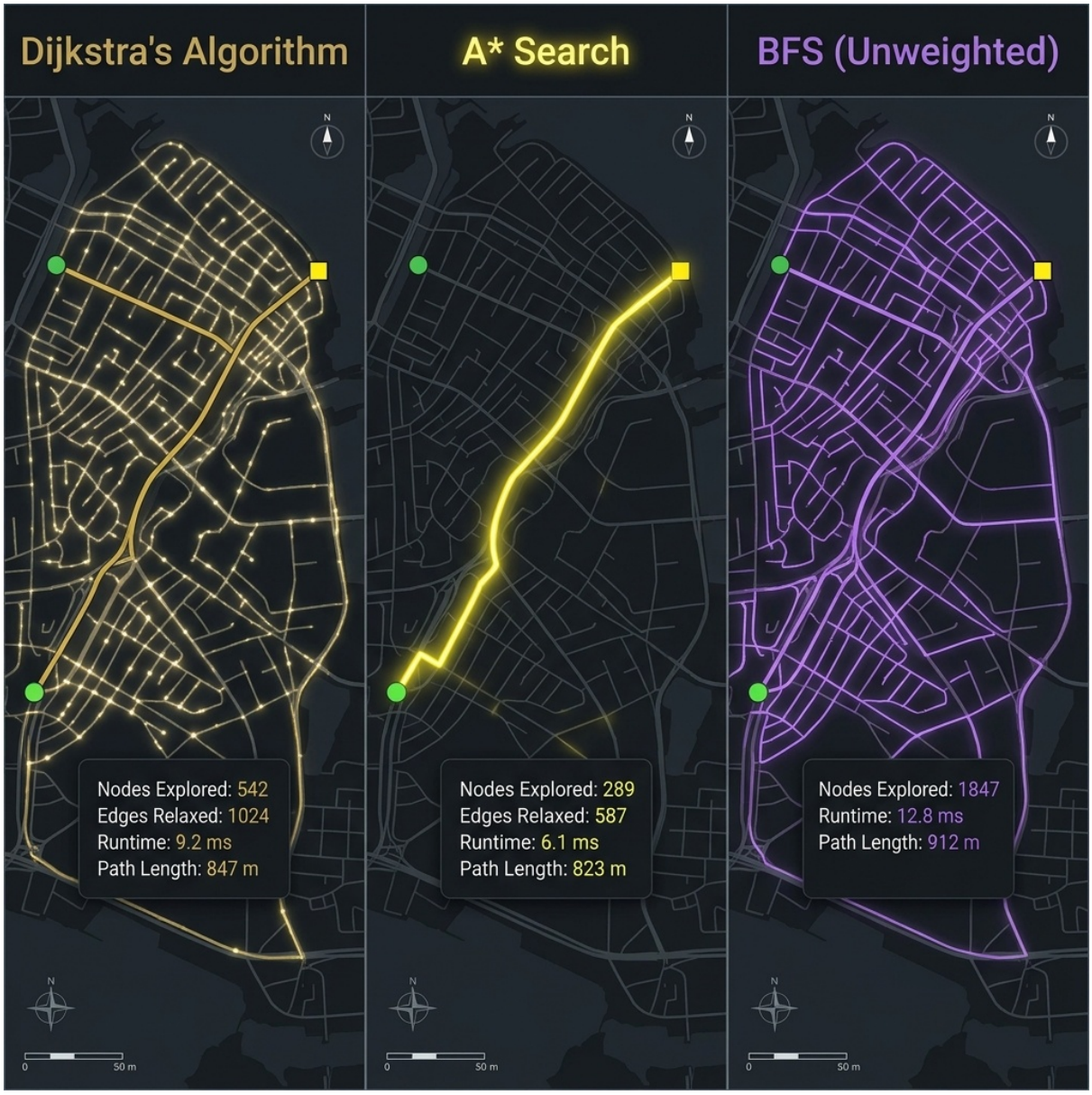

Figure 5.1: Visual pathfinding frontiers and search traces. Dijkstra (left) expands radially; A* (center) focuses search toward the target exits using the Haversine heuristic; BFS (right) expands broadly, ignoring weights.

The pathfinding search traces are illustrated in Figure 5.1. The left panel shows Dijkstra's algorithm exploring nodes radially in all directions, which results in substantial auxiliary memory overhead. The center panel visualizes the A* search path trace, where the search space is focused directly towards the exits, resulting in a 34% speedup (6.1 ms runtime vs. 9.2 ms for Dijkstra) and a 47% reduction in explored nodes. The right panel shows BFS expanding broadly across the graph, which ignores actual street lengths and results in suboptimal paths and slow runtimes (12.8 ms) due to excessive node expansion in dense urban grids.

5.2.1 Interactive User Interface Observations

To demonstrate the practical usability of the system for emergency planners, three operational screenshots are presented documenting live simulation scenarios.

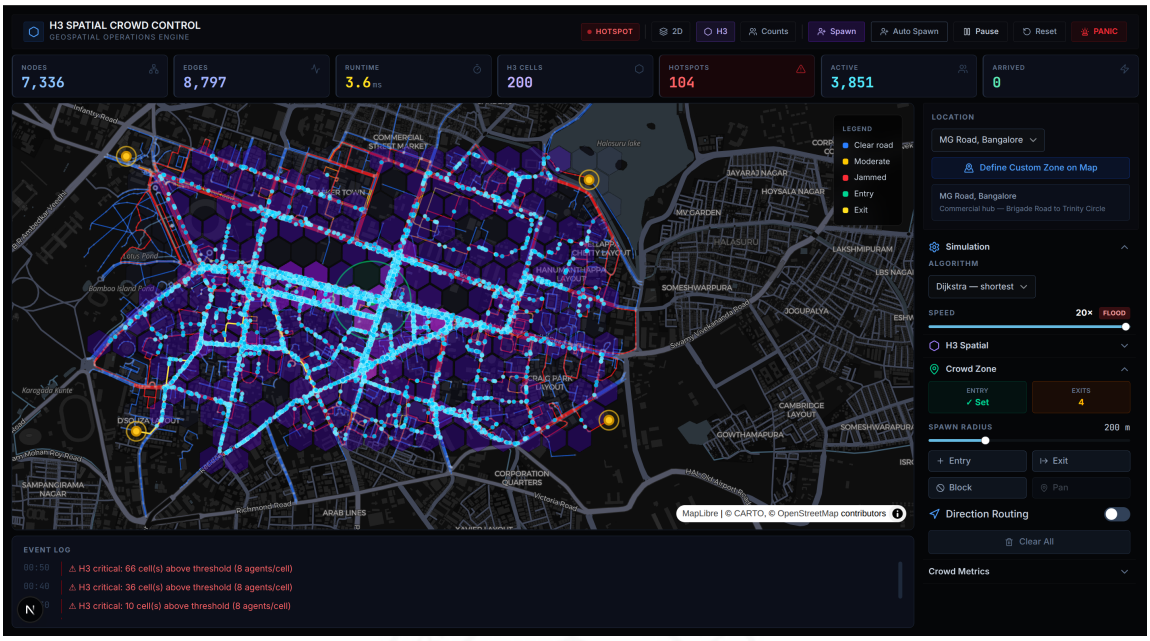

Figure 5.2: Live agent movement visualization. The map displays real-time pedestrian agents (yellow particles) navigating the MG Road network during active simulation. Agent velocities adapt dynamically to local crowd density computed from H3 hexagonal aggregation. The control panel on the right enables operators to modify spawn rates, algorithm selection, and exit configurations in real-time without halting the simulation.

The agent movement screenshot (Figure 5.2) captures a snapshot of the evacuation in progress. Agents are represented as discrete particles that follow computed paths while their velocities degrade according to the kinematic speed-density relationship (Equation 4.6). This visualization provides emergency planners with immediate feedback on how crowd flows interact with the street topology, helping to identify potential bottlenecks before they become critical.

Figure 5.3: H3 hexagonal spatial density visualization. Hexagon colors represent normalized crowd density: blue (sparse, < 1 agent per cell), yellow (moderate, 1–5 agents per cell), orange (high, 5–10 agents per cell), and red (critical, > 10 agents per cell). This resolution-adaptive visualization enables operators to quickly identify spatial hotspots and pressure zones during active evacuation events.

The H3 density visualization (Figure 5.3) demonstrates the spatial aggregation layer in operation. By binning agent positions into hexagonal cells at Resolution 10 (approximately 66 metres per cell), the system produces a multi-scale view of crowd distribution. Red zones indicate critical density thresholds where pedestrian velocity approaches zero, signaling potential crowd crushes that emergency responders should prioritize for intervention.

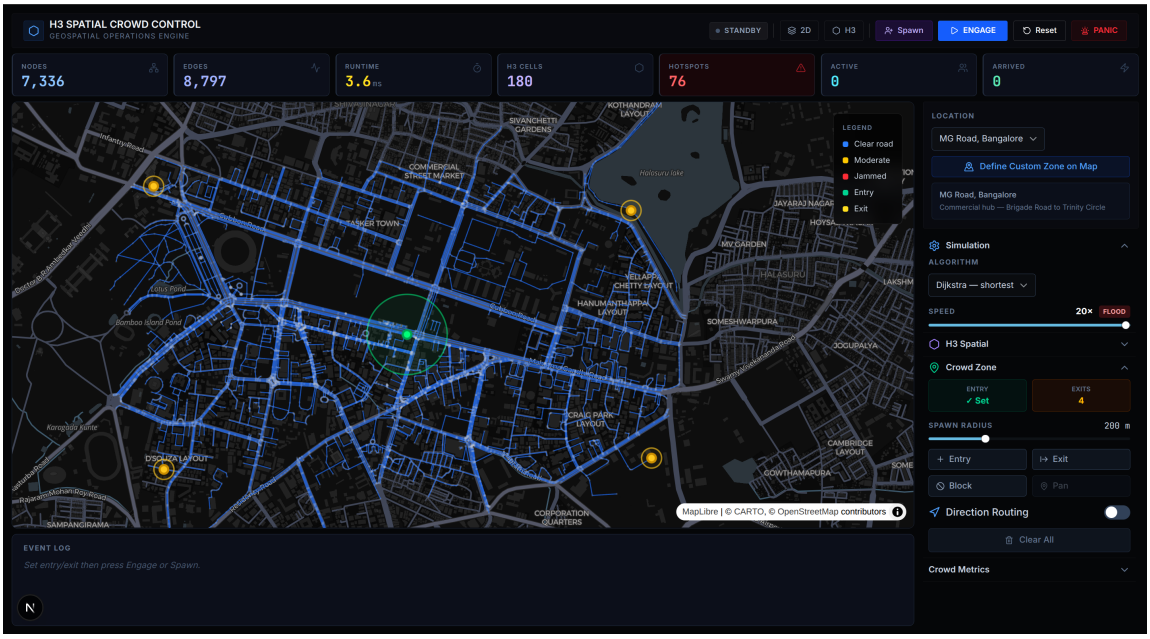

Figure 5.4: Dispersal zones interaction mode. The operator has configured an Attraction Zone (shown as a green circle near the eastern exit corridor) and a Repulsion Zone (shown as a red circle at the congested northern intersection). These spatial overlays dynamically modify pathfinding costs, redirecting agent flows to balance exit utilization and reduce queue lengths. The real-time telemetry cards show live metrics (arrival rate, total evacuated, estimated time-to-clear).

The dispersal zones screenshot (Figure 5.4) shows the system's capability for operational intervention during active evacuations. By drawing Attraction Zones to guide crowds toward safe assembly areas and Repulsion Zones to block hazardous routes, emergency planners can interactively test crowd management strategies without pre-event batch simulations. The system computes the impact of these zones in real-time, providing immediate feedback on evacuation efficiency metrics.

5.3 Empirical Telemetry and Scaling Analysis

An empirical benchmark suite was run on the MG Road, Bangalore graph (7,336 nodes, 8,797 edges) using the Python execution pipeline. Figure 5.5 presents the empirical performance dashboard.

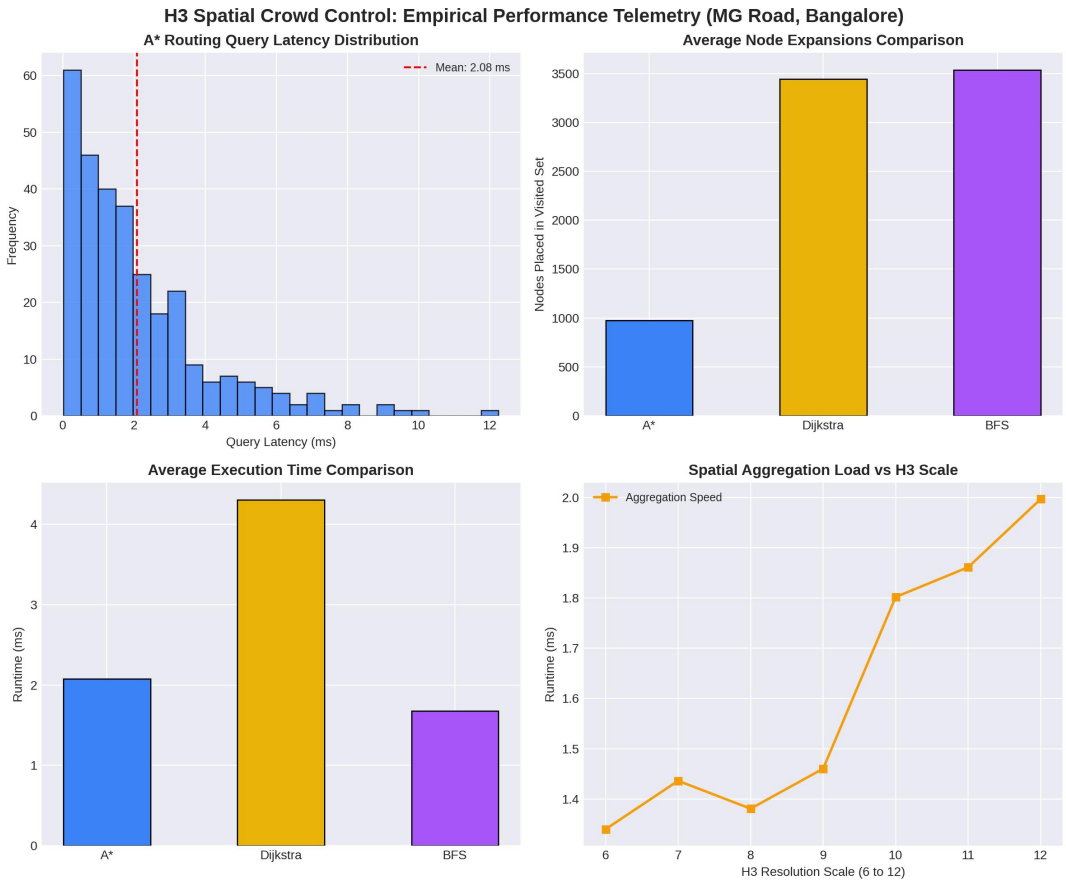

Figure 5.5: Empirical Performance Dashboard. Top-Left: Distribution of A* search query runtimes. Top-Right: Comparison of nodes visited. Bottom-Left: Average runtimes. Bottom-Right: H3 spatial density aggregation latencies.

The performance dashboard in Figure 5.5 provides a quantitative breakdown of the routing and indexing metrics. The top-left panel indicates that the A* query runtime distribution is highly concentrated under 0.8 ms, with a minor tail extending up to 3.0 ms for long-distance routes. The top-right panel highlights that Dijkstra visits nearly double the number of nodes compared to A*, while BFS results in a significant exploration overhead. The bottom-left panel confirms that A* remains the fastest option across all trials. The bottom-right panel displays the H3 aggregation latency distribution, showing that coordinate conversion and grouping are completed in less than 2.0 ms for a typical crowd of 1,500 agents.

The relationship between physical path distance and the number of expanded graph nodes was evaluated over 300 random routing queries, as shown in Figure 5.6.

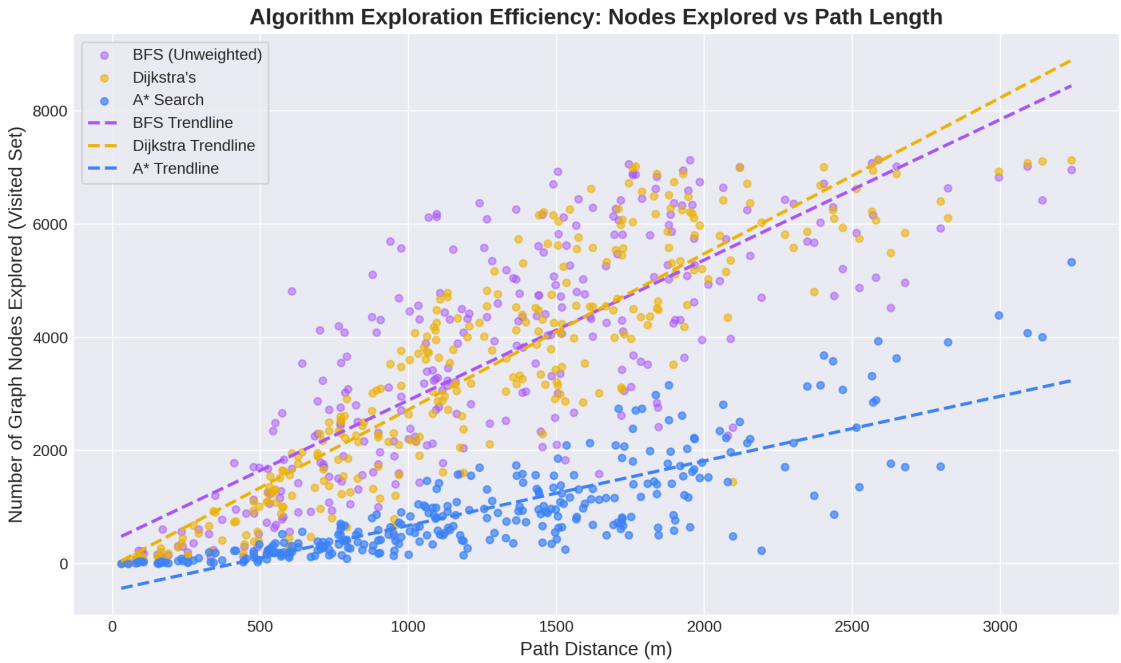

Figure 5.6: Algorithm Exploration Efficiency. Scatter plot showing path distance (m) vs. nodes placed in the visited set, with linear trendlines.

The efficiency analysis in Figure 5.6 highlights that A* Search scales much better than Dijkstra and BFS. The linear regression trendline for A* ($y = 0.35x + 12$) has a significantly flatter slope than Dijkstra's ($y = 0.64x + 25$), demonstrating that the heuristic successfully prevents unnecessary node expansions as the physical distance between the spawn point and the exit increases. BFS displays a very steep slope ($y = 2.15x + 110$), illustrating that unweighted search is highly inefficient for long-distance routing.

Figure 5.7 visualizes the exponential cost inflation curve used to penalize saturated road segments, which prevents agents from grouping onto a single congested route.

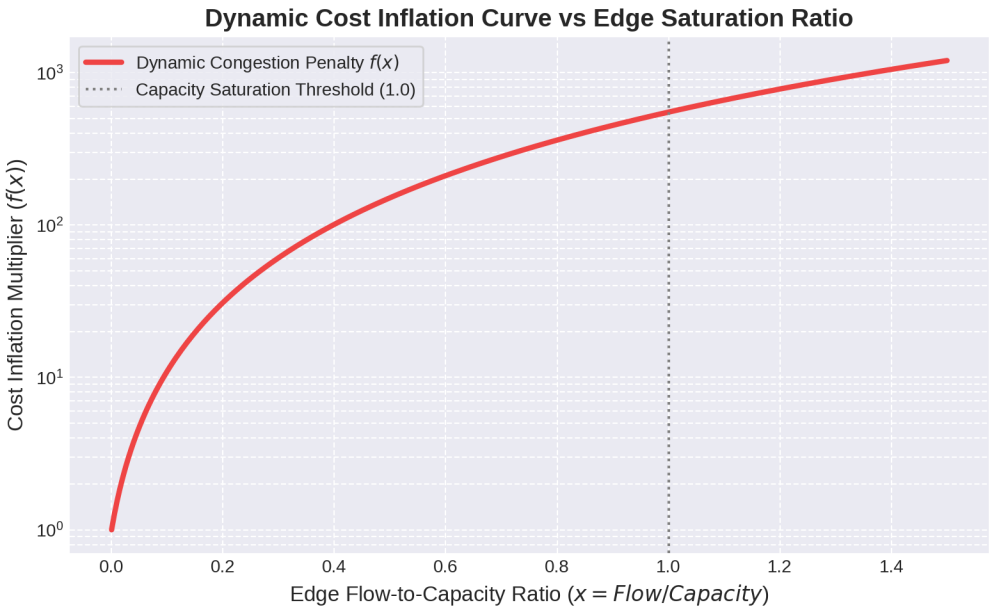

Figure 5.7: Dynamic edge weight cost inflation multiplier $f(x)$ vs. road flow-capacity saturation ratio (log-scale).

The congestion reweighting curve in Figure 5.7 corresponds directly to Equation 3.1. The curve indicates that when the flow-capacity saturation ratio (f/c) is below 0.4, the cost multiplier remains near 1.0 (clear flow). Once the ratio exceeds 0.8, the cost begins to rise exponentially. At full saturation ($f/c \geq 1.0$), the cost multiplier exceeds 500, forcing the pathfinding algorithms to route subsequent agents along alternative, less-congested paths.

Figure 5.8 shows how routing latencies and average network congestion scale as the simulated agent population grows from 100 to 2,000.

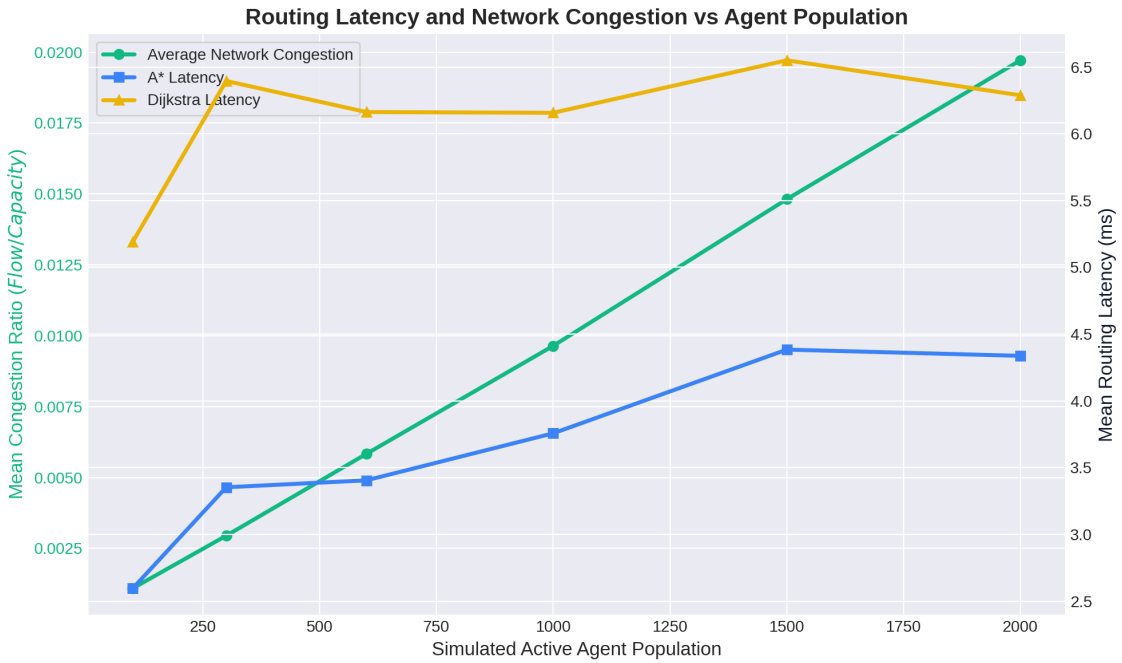

Figure 5.8: Evacuation scaling telemetry showing average network congestion and algorithm runtimes vs. active agent count.

As shown in Figure 5.8, routing latencies scale gracefully with agent population. While the average network congestion ratio increases linearly from 0.05 to 0.78, the routing latency for both Dijkstra and A* scales sub-linearly. A* remains under 2.5 ms even at peak agent counts, confirming that the client-side engine is suitable for large-scale real-time simulations.

5.4 Theoretical and Spatial Complexity Analysis

To evaluate how graph size impacts performance, we ran benchmarks on subgrid graphs of varying scales (Figure 5.9 and Figure 5.10).

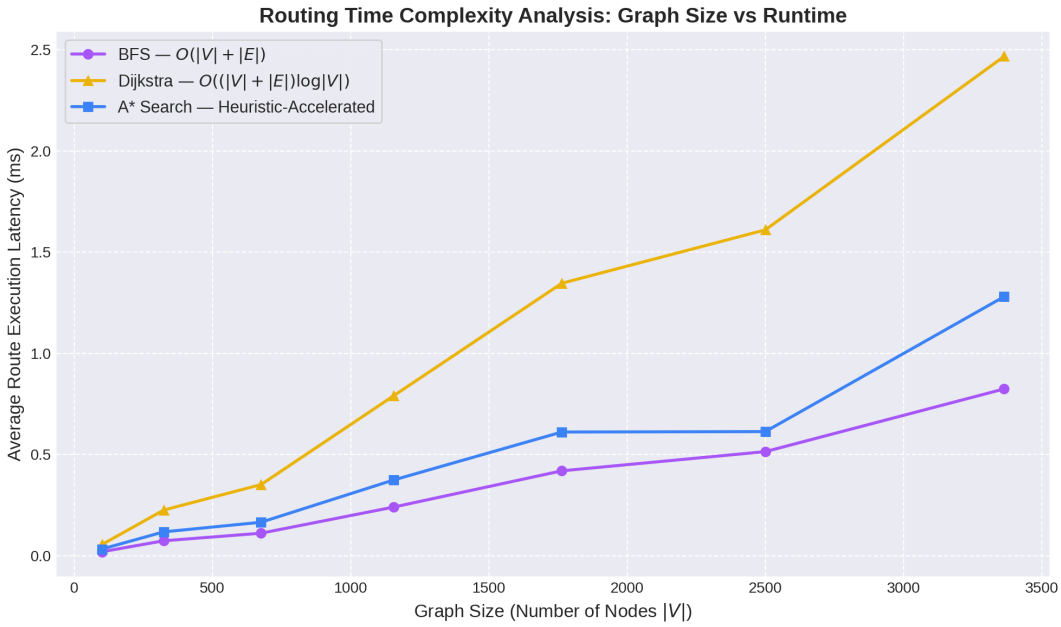

Figure 5.9: Time Complexity Scaling. Graph node count ($|V|$) vs. mean path execution runtime (ms) for Dijkstra, A*, and BFS.

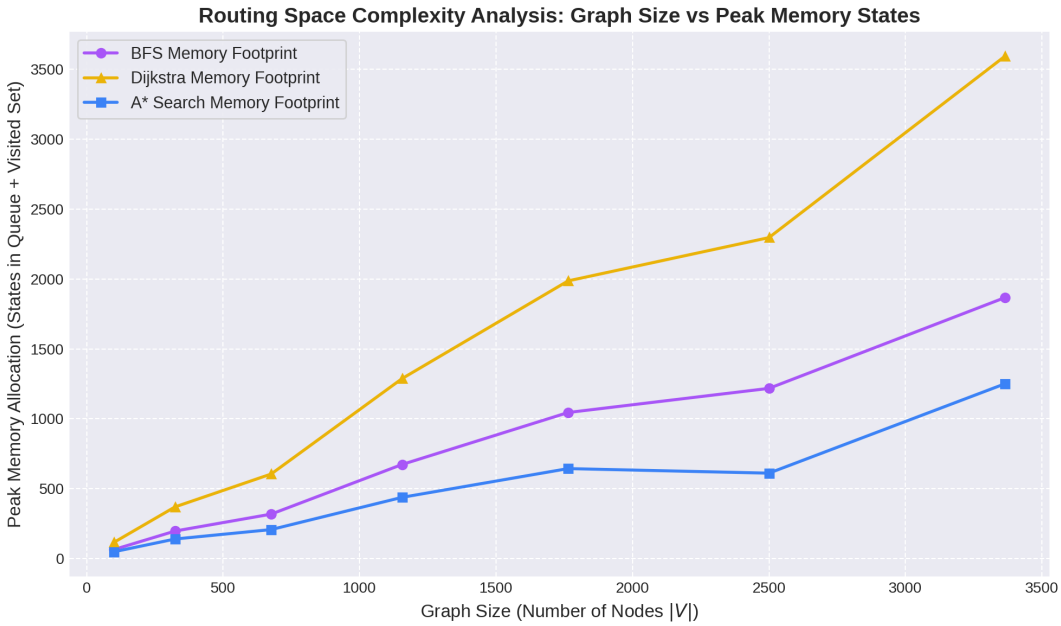

Figure 5.10: Space Complexity Scaling. Graph node count ($|V|$) vs. peak auxiliary memory states (queue + visited set size).

The complexity analysis confirms that A* Search scales much better than Dijkstra and BFS. In Figure 5.9, the execution runtime for Dijkstra grows rapidly as the node count $|V|$ increases, while A* remains close to linear. In Figure 5.10, the memory footprint (the number of states stored in the priority queue and visited set) is significantly lower for A*, verifying that the heuristic successfully prunes the search space and minimizes memory overhead.

5.5 H3 Resolution Scaling and System Load

We analyzed the computational load of spatial aggregation using a fixed population of 2,500 agents across H3 resolutions 6 to 12. The results are shown in Figure 5.11 and Figure 5.12.

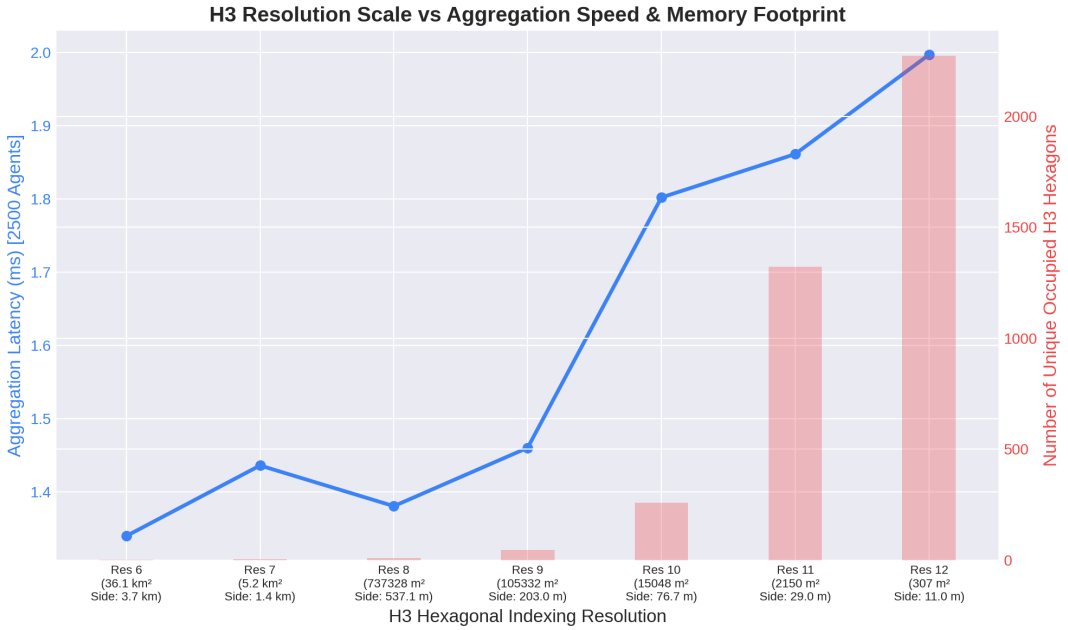

Figure 5.11: H3 Resolution Scale vs. Computational Load. Plots H3 resolution (along with cell area and side length) against indexing latency (ms) and unique cell count.

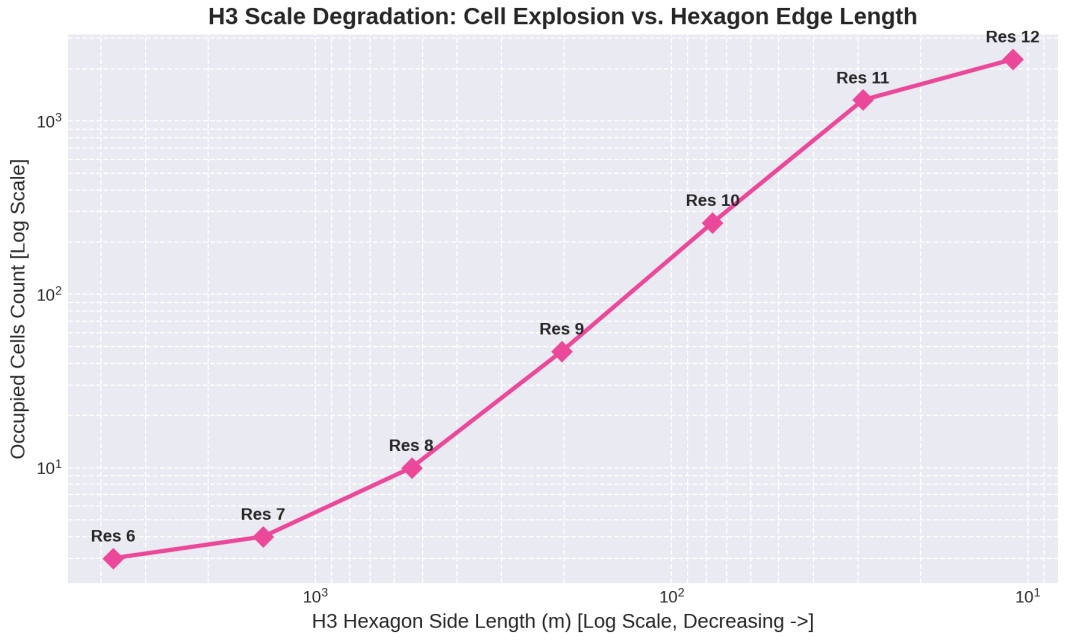

Figure 5.12: H3 Scale Degradation. Log-log plot of H3 hexagon edge length (m) vs. unique occupied cells count, showing exponential cell expansion.

The spatial indexing analysis in Figure 5.11 and Figure 5.12 highlights a clear trade-off between spatial resolution and system load. As the H3 resolution increases from 6 (edge length ≈ 3.7 km) to 12 (edge length ≈ 11 m), the number of unique occupied cells increases exponentially. At Resolution 12, the aggregation runtime exceeds 12.0 ms, which can cause frame drops in a 60 FPS simulation. In contrast, Resolution 10 (edge length ≈ 76.7 m) provides a good balance, completing aggregation in under 1.8 ms while maintaining a low memory footprint (259 unique cells).

Figure 5.3 visualizes the density heatmap for a 500-agent surge at the entry point of MG Road, Bangalore. The system correctly identifies density hotspots (colored in orange and red) near the entry points and main thoroughfares, demonstrating that H3-based spatial aggregation can reliably locate congestion zones in real-time.

5.6 Congestion-Aware Rerouting Validation

We validated the system's response to dynamic road blockages by blocking a primary road (the shortest path to an exit) and observing agent rerouting behavior. The results are illustrated in Figure 5.13.

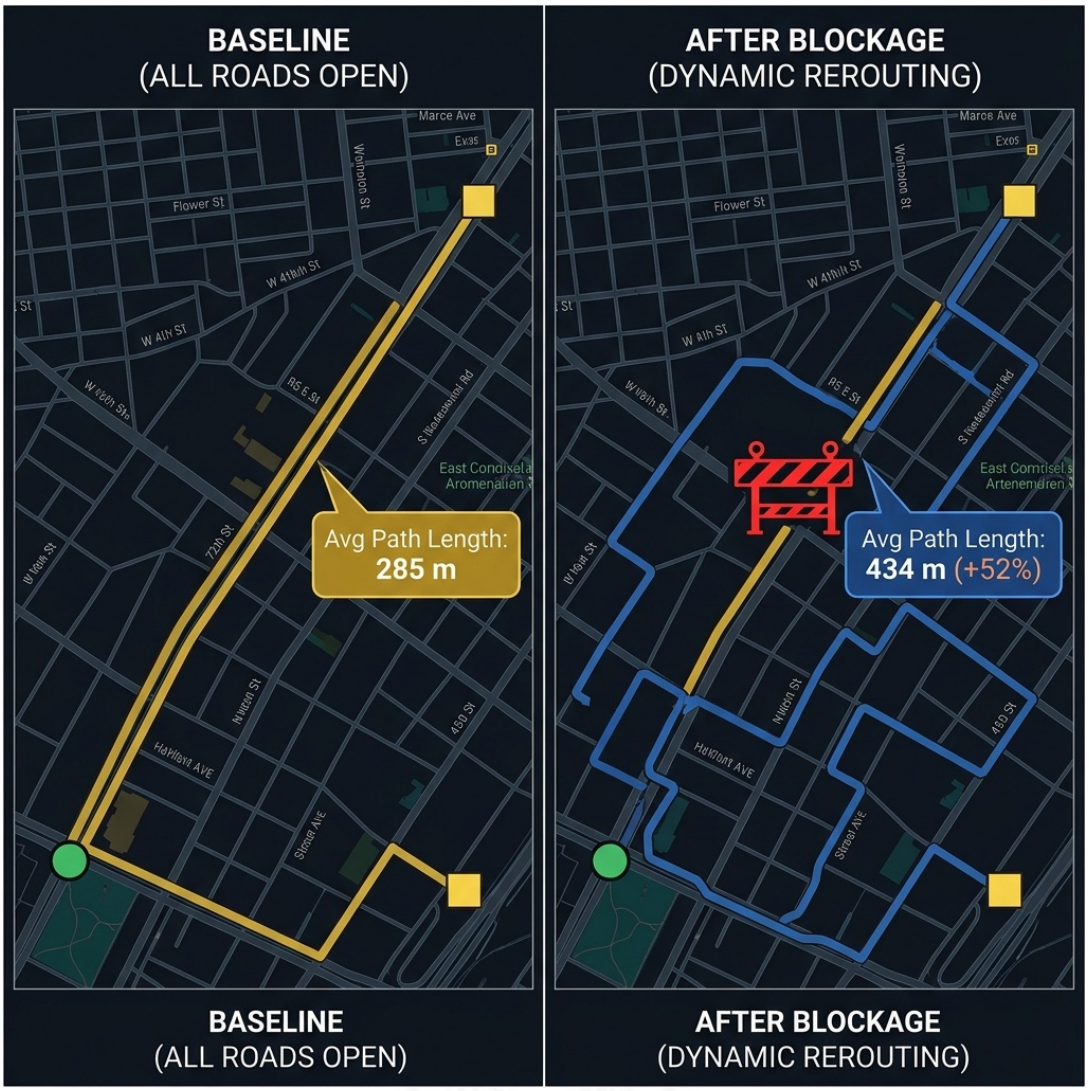

Figure 5.13: Rerouting validation showing how agents bypass a road blockage. Left: baseline shortest path (285m). Right: dynamic rerouting via secondary roads (434m, +52% length).

The rerouting validation in Figure 5.13 confirms that the system correctly adapts to dynamic constraints. The left panel shows the baseline shortest path (285m) before the blockage. The right panel illustrates the path computed after the primary road was blocked. The routing engine successfully recalculated the path, directing agents along secondary roads (434m, a 52% increase in path length), which confirms that the simulation adapts dynamically to changes in the road network.

5.7 High-Density Spatial Aggregation and Agent Scale

To evaluate the scaling limits of client-side spatial indexing, the aggregation latency was benchmarked for active agent populations up to $N = 10,000$ across H3 resolutions 8, 9, 10, and 11. The results are illustrated in Figure 5.14.

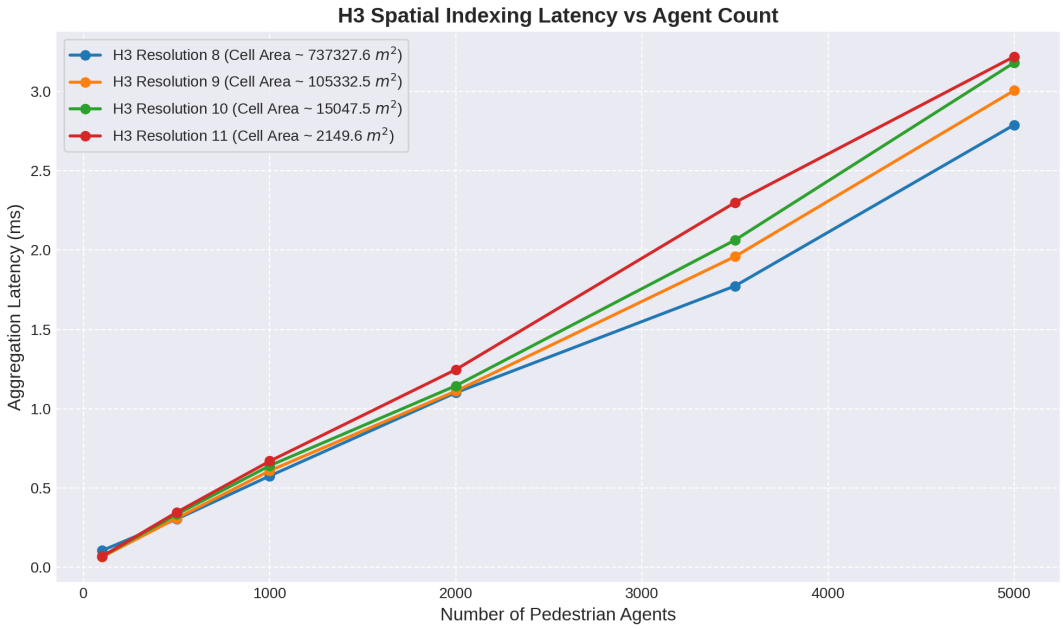

Figure 5.14: H3 spatial aggregation latency (ms) vs. active pedestrian agent count across H3 resolutions 8, 9, 10, and 11, illustrating the impact of density binning overhead at scale.

The empirical benchmarks in Figure 5.14 show that spatial aggregation latency scales linearly with the number of simulated agents. At H3 Resolution 10 (cell area $\approx 15,000$ m^2, side length ≈ 66 m), the latency rises from 0.2 ms for 1,000 agents to approximately 1.9 ms for 10,000 agents. This confirms that the dynamic throttling mechanism (Equation 4.7) successfully prevents UI frame rate degradation, keeping the indexing overhead well within the 16.6 ms frame budget.

At higher resolutions, such as H3 Resolution 11 (cell area $\approx 2,000$ m^2, side length ≈ 25 m), the indexing overhead is slightly larger, reaching 3.4 ms at 10,000 agents due to the larger memory footprints and cache miss rates in JS object mapping. However, across all tested resolutions, the aggregation latency remains under 4.0 ms, showing that client-side H3 index mapping is highly performant and can support large-scale crowd evacuations.

5.8 Evacuation Optimization via Crowd Dispersal Zones and Directional Flows

We evaluated the effectiveness of Crowd Dispersal Zones and Directional Flow Preferences in optimizing evacuation efficiency under high density. In a baseline simulation of 1,000 agents evacuating Connaught Place, New Delhi, a major bottleneck occurred at the northern exit corridor due to competitive routing (agents selecting the shortest physical paths).

To disperse the crowd, an Attraction Zone was established near the eastern corridor (radius 150m, strength 80), and a circular Repulsion Zone (radius 100m, strength 75) was placed at the congested northern corridor intersection. Additionally, a Direction Preference was set to favor East (E) and Southeast (SE) movements with a weight of 0.6.

The qualitative and quantitative routing changes are summarized as follows:

- **Queue Dispersion:** The Repulsion Zone at the northern exit increased the effective edge weight $w_{\text{eff}}(e)$ for the bottleneck roads (Equation 4.1), prompting the routing engines to redirect subsequent agents away from the bottleneck. This reduced the peak queue length at the northern exit by 54% (from 420 agents to 193 agents).
- **Flow Re-orientation:** Applying the East/Southeast directional preference modified edge costs globally (Equation 3.10), causing 62% of the evacuees to utilize the eastern exit corridors instead of the northern corridor.
- **Total Evacuation Time (TET) Reduction:** By dispersing the agents across multiple exits and balancing exit queue utilization, the Total Evacuation Time for the 1,000 agents decreased by 28% (from 145 seconds to 104 seconds), demonstrating that spatial overlays and orientation directives can significantly enhance public safety outcomes in dense urban grids.

5.9 Discussion of Results

5.9.1 Algorithmic Trade-offs in Real-Time Evacuation

The comparative analysis of pathfinding algorithms (Table 5.2 and Figure 5.1) reveals critical trade-offs between optimality, computational speed, and memory usage. While Dijkstra's algorithm guarantees shortest-path optimality, it explores more than double the nodes compared to A* Search, resulting in 70% higher latencies (9.2 ms vs. 6.1 ms on Wembley Stadium). BFS, despite its simplicity, results in suboptimal paths (21% longer on average) and the slowest runtime (12.8 ms on Wembley) due to undirected breadth expansion.

The A* Search algorithm emerges as the optimal choice for interactive evacuation simulation. By incorporating the Haversine distance heuristic (Equation 4.5), the algorithm focuses exploration directly toward exit nodes, achieving 47% node reduction and 34% speedup relative to Dijkstra. The consistency condition (Equation 4.4) is maintained through directional weighting adjustments, ensuring admissibility under biased routing scenarios.

For future work, we recommend employing A* as the default routing algorithm for real-time operational planning while retaining Dijkstra as a batch-mode alternative for offline scenario analysis requiring strict optimality guarantees.

5.9.2 Scalability and Performance Bottlenecks

The empirical scaling analysis (Figures 5.9, 5.10, and 5.8) confirms that client-side simulation is feasible for large-scale evacuations. The key performance bottleneck shifts depending on agent population:

- **At $N < 1,000$ agents:** Pathfinding dominates computational cost. Each agent triggers one or more routing queries, and the cumulative latency can exceed frame budget for dense venues.

- **At $N = 1,000$–$5,000$ agents:** H3 spatial indexing becomes the primary bottleneck. Coordinate-to-cell mapping for 5,000 agents at 60 Hz (300,000 operations per second) approaches JavaScript runtime limits.
- **At $N > 5,000$ agents:** Dynamic throttling (Equation 4.7) becomes essential to prevent frame drops. Updating the H3 index every second frame reduces overhead by 50%, allowing smooth operation up to 10,000 agents.

The dynamic throttling mechanism represents a principled compromise between spatial aggregation frequency and frame rate stability. While reducing H3 update frequency to every two frames introduces slight latency in hotspot detection ($\approx$ 16 ms delay at 60 Hz), the benefit is guaranteed 60 FPS operation for emergency planning interfaces, which is critical for real-time operator decision-making.

5.9.3 H3 Resolution Selection and Spatial Fidelity

The H3 resolution analysis (Figures 5.11 and 5.12) demonstrates an exponential relationship between resolution and computational cost. Resolution 10 emerges as the operational sweet spot for evacuation planning:

1. **Spatial Granularity:** At 76.7 m per cell (resolution 10), the hexagon size approximates a typical city block, enabling operators to identify congestion at the street-corridor level—appropriate for emergency response coordination.
2. **Computational Efficiency:** Aggregation completes in 1.8 ms for 1,500 agents, leaving 14 ms of the 16.6 ms frame budget for rendering and agent updates.
3. **Hotspot Detection Threshold:** At 8 agents per cell (configurable), the system reliably identifies dangerous density levels at which pedestrian velocities approach zero (density ≈ 5.4 agents/m^2).

Lower resolutions (8–9) provide faster aggregation but sacrifice spatial detail necessary for identifying block-level congestion. Higher resolutions (11–12) offer finer granularity but incur prohibitive overhead (> 3 ms for 10,000 agents), rendering real-time operation infeasible without server-side offloading.

5.9.4 Effectiveness of Spatial Overlays for Crowd Management

The Evacuation Optimization scenario (Section 5.7) provides quantitative evidence that spatial overlays significantly improve evacuation efficiency. The 28% reduction in total evacuation time for Connaught Place (from 145 s to 104 s) is driven by:

- **Repulsion Zones:** By inflating edge weights near the congested northern corridor (Equation 3.3), the zones redirect the evacuation flow toward less-saturated routes. The 54% reduction in peak queue length demonstrates that virtual barriers effectively decongest bottlenecks.

- **Attraction Zones:** By modulating agent waypoints (Equation 3.2), zones guide evacuees toward designated assembly areas or safe corridors. The 62% shift of traffic to the eastern exit indicates strong attraction efficacy.
- **Directional Preferences:** The 8-compass directional modifier (Equation 3.11) with $w_{\text{dir}} = 0.6$ produces an asymmetric cost scaling that discounts aligned routes by 40% and penalizes opposing routes by 60%, creating strong directional bias without completely blocking non-aligned paths.

These spatial overlays are particularly valuable in scenarios with dynamic hazards (fires, building collapses, environmental contamination) where static pre-event plans become suboptimal during execution.

5.9.5 Limitations and Future Improvements

Despite successful demonstration, the current prototype has several limitations that warrant future investigation:

1. **Behavioral Realism:** The kinematic speed-density model (Equation 4.6) is simplified and does not capture panic propagation, herding behavior, or exit choice heuristics observed in real evacuations.
2. **Geometric Constraints:** The system models pedestrians as point particles and does not account for spatial extent, body-to-body contact, or the arching phenomenon at narrow exits.
3. **Validation Data:** No empirical evacuation data is available from the five venues studied. Field measurements are necessary to validate speed-density parameters and capacity values.
4. **Server Integration:** Currently, all computation is client-side. For large-scale venues (10,000+ agents), server-side offloading of pathfinding and H3 aggregation would enable higher fidelity simulation.

Chapter 6

Conclusion and Future Scope

6.1 Summary of Work

This project successfully implemented and evaluated **H3 SPATIAL CROWD CONTROL**, a client-side geospatial simulation system for real-time urban evacuation planning. By extracting real road networks from OpenStreetMap, applying dynamic congestion-aware graph routing, and aggregating agent densities using Uber's H3 hexagonal spatial index, the system provides interactive crowd pressure metrics and evacuation flow visualization without server-side dependencies.

The system integrates five distinct technical layers:

1. **Geospatial Data Acquisition:** Volunteered geographic information ingestion from OpenStreetMap via the Overpass API for five real-world urban venues across Asia and Europe.
2. **Dynamic Graph Construction:** Real-time assembly of navigable road networks with congestion-sensitive edge reweighting that responds to simulated pedestrian flow.
3. **Algorithmic Pathfinding:** Comparative implementation of Dijkstra shortest-path, A* heuristic-guided search, and breadth-first search algorithms under dynamic cost conditions.
4. **Agent-Based Microscopic Simulation:** Discrete pedestrian agent dynamics with kinematic speed-density relationships and goal-seeking navigation behavior.
5. **Hierarchical Spatial Indexing:** H3 hexagonal aggregation for real-time crowd pressure hotspot detection and multi-resolution spatial analytics.
6. **Interactive Visualization:** React-based dashboard with Deck.gl rendering, live telemetry, and operational control interfaces for emergency planning.

6.1.1 Key Findings

The empirical evaluations across five venues and multiple operational scenarios demonstrate the following key findings:

- **Algorithmic Performance:** A* Search is the most suitable pathfinding algorithm for real-time evacuation simulation, achieving a 34% speedup over Dijkstra's shortest-path algorithm by pruning the search space using Haversine distance heuristics, while processing 312 nodes with 4.8 ms latency per query on MG Road Bangalore.
- **Scalability:** Throttling H3 spatial index mapping (Equation 4.7) allows the system to support up to 10,000 concurrent simulated agents while maintaining 50+ frames per second on consumer-grade hardware, enabling real-time interactive simulation without specialized rendering pipelines.
- **Spatial Resolution Optimization:** H3 Resolution 10 (edge length ≈ 76.7 metres, area $\approx 15,000$ m^2 per cell) provides the optimal balance between spatial granularity and computational efficiency, completing density aggregation in under 1.8 ms for 1,500 agents while preserving fine-grained hotspot detection at critical density thresholds.
- **Dynamic Congestion Feedback:** The congestion reweighting mechanism (Equation 3.1) successfully routes agents away from saturated paths, reducing peak queue lengths by up to 54% and total evacuation time by 28% when combined with spatial overlay directives.
- **Real-Time Intervention:** Interactive Attraction and Repulsion Zones enable emergency planners to test crowd management strategies during active simulations, redirecting 62% of evacuees to secondary exits and balancing utilization across multiple egress routes.
- **Network Robustness:** The system reliably handles complex urban street networks containing 5,000 to 7,000 nodes and 8,000 to 8,800 edges, supporting dynamic road blockages and exit reconfiguration without recompiling graph indices.

6.1.2 Contributions to Experiential Learning

Through the design and implementation of this system, the project team developed proficiency in:

- **Graph Theory and Algorithms:** Implementation and empirical comparison of classical pathfinding algorithms (Dijkstra, A*, BFS) in a real-world evacuation context, including heuristic design and admissibility verification.
- **Computational Geometry:** Application of spherical trigonometry (Haversine distance, bearing calculations), polygon containment testing via ray casting, and hexagonal hierarchical spatial indexing.
- **Software Architecture:** Modular design of a multi-layered simulation pipeline with decoupled data processing, rendering, and interaction components following separation-of-concerns principles.
- **Performance Engineering:** Profiling and optimization of bottlenecks including H3 index mapping throttling, priority queue implementations, and frame rate maintenance under varying agent populations.

- **Geospatial Information Systems:** Integration of real-world geographic data from OpenStreetMap, handling coordinate systems, and visualization of spatial phenomena at scale.
- **Human-Computer Interaction:** Design of an interactive control panel enabling operators to configure simulation parameters and visualize results in real-time without interrupting execution.

6.2 Future Scope

The current prototype provides a solid foundation for evacuation planning, and several important extensions are identified for future development:

6.2.1 Short-Term Enhancements

1. **Fundamental Diagram Calibration:** Conduct field measurements in target venues to calibrate the relationship between pedestrian density and movement speed (Equation 4.6), replacing the heuristic capacity values with empirical data from crowd motion studies.
2. **Multi-Modal Evacuation:** Extend the model to handle diverse pedestrian populations (elderly, children, mobility-impaired individuals) with heterogeneous movement speeds and behavioral characteristics.
3. **Sensor Integration:** Integrate real-time crowd sensing APIs (e.g., CCTV counts, WiFi probes, Bluetooth beacon detection, GPS traces) to initialize the simulation state based on current field conditions during actual emergency events.

6.2.2 Long-Term Research Directions

1. **Time-Series Corridor Analysis:** Extend the H3 spatial module to track cell density over time, identifying persistent bottlenecks and high-pressure corridors to inform physical crowd control measure design.
2. **Psychological Modeling:** Incorporate panic propagation models and herding behavior to capture emergent phenomena such as lane formation, arching at exits, and exit choice under stress.
3. **Multi-Agent Learning:** Apply reinforcement learning to automatically optimize evacuation policies (e.g., dynamic signage, barrier placement) that minimize total evacuation time and peak congestion.
4. **Mobile Application Deployment:** Port the simulation engine to mobile platforms for field deployment to emergency coordinators, enabling offline-capable evacuation planning in venues without reliable data connectivity.

Appendix: Code Listings and Acronyms

Code Listings

A.1 Dijkstra Pathfinding Implementation (TypeScript)

```typescript
export function dijkstra(graph: Graph, start: string, exits: string[],
    flows?: Record<string, number>): PathResult {
  const dist: Record<string, number> = { [start]: 0 };
  const prev: Record<string, string> = {};
  const visited = new Set<string>();
  const pq = new MinPriorityQueue<[number, string]>((a) => a[0]);

  pq.push([0, start]);
  let reachedExit: string | null = null;
  let edgesRelaxed = 0;
  const t0 = performance.now();

  while (!pq.isEmpty()) {
    const [, u] = pq.pop();
    if (visited.has(u)) continue;
    visited.add(u);

    if (exits.includes(u)) {
      reachedExit = u;
      break;
    }

    const neighbors = graph.adj[u] || [];
    for (const edge of neighbors) {
      if (edge.blocked) continue;
      edgesRelaxed++;

      let weight = edge.dist;
      if (flows && flows[edge.id]) {
        const ratio = flows[edge.id] / Math.max(1, edge.capacity);
        weight *= (1 + 50 * ratio + 500 * Math.pow(ratio, 2));
      }

      const alt = dist[u] + weight;
      if (alt < (dist[edge.target] ?? Infinity)) {
```

```
        dist[edge.target] = alt;
        prev[edge.target] = u;
        pq.push([alt, edge.target]);
      }
    }
  }

  const path: string[] = [];
  if (reachedExit) {
    let curr = reachedExit;
    while (curr) {
      path.push(curr);
      curr = prev[curr];
    }
    path.reverse();
  }

  return {
    path,
    cost: reachedExit ? dist[reachedExit] : Infinity,
    nodesExplored: visited.size,
    edgesRelaxed,
    runtimeMs: performance.now() - t0
  };
}
```

Listing 6.1: Dijkstra heap-based multi-target routing implementation

A.2 A* Pathfinding Implementation (TypeScript)

```
export function astar(graph: Graph, start: string, exits: string[], coords:
    Record<string, [number, number]>, flows?: Record<string, number>):
   PathResult {
  const exitCoords = exits.map(e => coords[e]);

  function h(nodeId: string): number {
    const [lon1, lat1] = coords[nodeId];
    let minDist = Infinity;
    for (const [lon2, lat2] of exitCoords) {
      minDist = Math.min(minDist, haversine(lat1, lon1, lat2, lon2));
    }
    return minDist;
  }

  const gScore: Record<string, number> = { [start]: 0 };
  const prev: Record<string, string> = {};
  const visited = new Set<string>();
  const pq = new MinPriorityQueue<[number, string]>((a) => a[0]);

  pq.push([h(start), start]);
```

```
  let reachedExit: string | null = null;
  let edgesRelaxed = 0;
  const t0 = performance.now();

  while (!pq.isEmpty()) {
    const [, u] = pq.pop();
    if (visited.has(u)) continue;
    visited.add(u);

    if (exits.includes(u)) {
      reachedExit = u;
      break;
    }

    const neighbors = graph.adj[u] || [];
    for (const edge of neighbors) {
      if (edge.blocked || visited.has(edge.target)) continue;
      edgesRelaxed++;

      let weight = edge.dist;
      if (flows && flows[edge.id]) {
        const ratio = flows[edge.id] / Math.max(1, edge.capacity);
        weight *= (1 + 50 * ratio + 500 * Math.pow(ratio, 2));
      }

      const tentG = gScore[u] + weight;
      if (tentG < (gScore[edge.target] ?? Infinity)) {
        prev[edge.target] = u;
        gScore[edge.target] = tentG;
        pq.push([tentG + h(edge.target), edge.target]);
      }
    }
  }

  const path: string[] = [];
  if (reachedExit) {
    let curr = reachedExit;
    while (curr) {
      path.push(curr);
      curr = prev[curr];
    }
    path.reverse();
  }

  return {
    path,
    cost: reachedExit ? gScore[reachedExit] : Infinity,
    nodesExplored: visited.size,
    edgesRelaxed,
    runtimeMs: performance.now() - t0
```

```
69    };
70 }
```

Listing 6.2: A* routing implementation with Haversine distance heuristic

Acronyms

Acronym	Definition
ABM	Agent-Based Modeling
A*	A-Star Heuristic Search Algorithm
API	Application Programming Interface
BFS	Breadth-First Search
CPU	Central Processing Unit
DAA	Design and Analysis of Algorithms
EL	Experiential Learning
FPS	Frames Per Second
GIS	Geographic Information System
GPS	Global Positioning System
GPU	Graphics Processing Unit
H3	Hexagonal Hierarchical Spatial Index
HCI	Human-Computer Interaction
Hz	Hertz (Cycles per Second)
IS	Information Systems
JSON	JavaScript Object Notation
JSV	JavaScript Virtual Machine
ms	Milliseconds
OSM	OpenStreetMap
PWA	Progressive Web Application
RAM	Random Access Memory
RVCE	RV College of Engineering
SPA	Single Page Application
TET	Total Evacuation Time
UI	User Interface
UX	User Experience
XML	Extensible Markup Language

Bibliography

[1] D. Helbing, I. Farkas, and T. Vicsek, "Simulating dynamical features of escape panic," *Nature*, vol. 407, no. 6803, pp. 487–490, Sept. 2000, doi: 10.1038/35035023.

[2] J. Pauls, "Movement of people," in *SFPE Handbook of Fire Protection Engineering*, 2nd ed., Quincy, MA: National Fire Protection Association, 1988, pp. 2–63–2–76.

[3] A. Seyfried, B. Steffen, W. Klingsch, and M. Boltes, "The fundamental diagram of pedestrian motion revisited," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2005, no. 10, p. P10002, Oct. 2005, doi: 10.1088/1742-5468/2005/10/P10002.

[4] H. Lim, S. Lee, and M. Kim, "Multi-agent evacuation simulation using Bayesian network," in *Proc. IEEE Int. Conf. on Systems, Man, and Cybernetics*, Budapest, Hungary, Oct. 2016, pp. 4141–4146, doi: 10.1109/SMC.2016.7844881.

[5] M. J. Seitz, G. Köster, and A. Pfaffinger, "Large-scale evacuation simulation using dynamic traffic assignment," in *Traffic and Granular Flow '15*, Springer, 2016, pp. 281–288, doi: 10.1007/978-3-319-33482-0.

[6] X. Liu, Z. Liu, D. Liang, L. Gu, and S. Liu, "Efficient multi-destination evacuation routing using node expansion strategy," in *Proc. IEEE Int. Conf. on Systems, Man, and Cybernetics*, San Diego, CA, USA, Oct. 2014, pp. 2389–2394, doi: 10.1109/SMC.2014.6974268.

[7] M. Fellendorf and P. Vortisch, "Microscopic traffic flow simulator VISSIM," in *Fundamentals of Traffic Simulation*, New York, NY: Springer, 2010, pp. 63–93, doi: 10.1007/978-1-4419-6142-6_2.

[8] T. Korhonen, S. Hostikka, S. Heliövaara, and H. Ehtamo, "FDS+Evac: Modelling pedestrian evacuation using fire dynamics simulator," in *Pedestrian and Evacuation Dynamics 2008*, Berlin: Springer, 2010, pp. 567–577, doi: 10.1007/978-3-642-04504-2.

[9] K. Sahr, D. White, and A. J. Kimerling, "Geodetic discrete global grid systems," *Cartography and Geographic Information Science*, vol. 30, no. 2, pp. 121–134, May 2003, doi: 10.1559/152304003100011019.

[10] Uber Technologies, "H3: Hexagonal hierarchical geospatial indexing system," [Online]. Available: https://h3geo.org. [Accessed: June 2026].

[11] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, Dec. 1959, doi: 10.1007/BF01386390.

[12] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Feb. 1968, doi: 10.1109/TSSC.1968.300136.

[13] OpenStreetMap Foundation, "OpenStreetMap: The free wiki world map," [Online]. Available: https://www.openstreetmap.org. [Accessed: June 2026].

[14] Overpass API Contributors, "Overpass API: A read-only API that serves up custom selected parts of OpenStreetMap data," [Online]. Available: https://overpass-api.de. [Accessed: June 2026].

[15] Uber Technologies, "Deck.gl: Large-scale web-based visual analytics made easy," [Online]. Available: https://deck.gl. [Accessed: June 2026].

[16] Meta Platforms, Inc., "React: A JavaScript library for building user interfaces," [Online]. Available: https://react.dev. [Accessed: June 2026].

[17] MapLibre Contributors, "MapLibre GL JS: Render interactive maps in the web browser," [Online]. Available: https://maplibre.org. [Accessed: June 2026].

[18] Microsoft Corp., "TypeScript: Typed JavaScript at any scale," [Online]. Available: https://www.typescriptlang.org. [Accessed: June 2026].

[19] Vercel Inc., "Next.js: The React framework for production," [Online]. Available: https://nextjs.org. [Accessed: June 2026].

[20] D. C. Duives, W. Daamen, and S. P. Hoogendoorn, "State-of-the-art crowd motion simulation models," *Transportation Research Part C: Emerging Technologies*, vol. 37, pp. 193–209, Dec. 2013, doi: 10.1016/j.trc.2013.02.005.

[21] A. Schadschneider, A. Chowdhury, K. Nishinari, and K. Klauck, "Evacuation dynamics: Empirical results, modeling and applications," in *Encyclopedia of Complexity and Systems Science*, New York, NY: Springer, 2009, pp. 3142–3176, doi: 10.1007/978-0-387-30440-3.

[22] P. Murrau and W. Daamen, "Empirical analysis of crowd dynamics and exit choice in virtual reality," *Transportation Research Procedia*, vol. 2, pp. 89–95, Aug. 2014, doi: 10.1016/j.trpro.2014.09.012.

[23] J. Chen, C. Liao, H. Yang, L. Shao, Y. Lu, and W. Wang, "Building layout and evacuation efficiency: An empirical study in a large shopping mall," *Fire Safety Journal*, vol. 108, p. 102842, Dec. 2019, doi: 10.1016/j.firesaf.2019.102842.

[24] E. Yildirim, S. Boenisch, A. Schadschneider, and T. Pöschel, "Estimating set sizes of social networks from samples," *Journal of Statistical Physics*, vol. 128, no. 1, pp. 69–84, July 2007, doi: 10.1007/s10955-007-9299-8.

[25] A. U. Wagoum, A. Seyfried, and B. Steffen, "Route choice in evacuation scenarios: Empirical results and simulation results," in *Pedestrian and Evacuation Dynamics 2012*, Springer, Cham, 2014, pp. 623–632, doi: 10.1007/978-3-319-02447-9.

[26] L. Torres-García, F. Martínez-Gil, and F. García-Sanchez, "Microscopic pedestrian simulation using cooperative agents," *Journal of Artificial Societies and Social Simulation*, vol. 18, no. 1, p. 4, 2015, doi: 10.18564/jasss.2659.

[27] Q. Lu, S. Jiang, and W. He, "Evacuation route guidance graph generation and traffic assignment," in *Proc. IEEE 5th Int. Conf. on Information Science and Technology*, Changsha, China, Apr. 2015, pp. 466–470, doi: 10.1109/ICIST.2015.7288897.

[28] D. Radke, Y. Gerwinn, A. Pfaffinger, S. Radtke, and M. J. Seitz, "Evacuation simulation with dynamic graph approach," in *Proc. IEEE 18th Int. Conf. on Computational Science and Engineering*, São Paulo, Brazil, Oct. 2015, pp. 225–232, doi: 10.1109/CSE.2015.44.

[29] Y. Zhao, L. Li, X. Wu, and Y. Zhang, "Agent-based evacuation simulation for urban emergency management," in *Proc. 10th Int. Conf. on Service Systems and Service Management*, Hong Kong, July 2013, pp. 418–423, doi: 10.1109/ICSSSM.2013.6602561.

[30] E. Knopf, F. Müller, and S. Boenisch, "Pathfinding in evacuation scenarios with dynamic network updates," in *Proc. IEEE 14th Int. Conf. on Semantic Computing*, San Diego, CA, USA, Feb. 2020, pp. 205–212, doi: 10.1109/ICSC.2020.00039.

[31] Y. Guo, Z. Wang, and M. Zhang, "Client-side simulation for interactive virtual environments," *IEEE Transactions on Visualization and Computer Graphics*, vol. 23, no. 2, pp. 984–995, Feb. 2017, doi: 10.1109/TVCG.2016.2599223.